CS 422 Assignment 2

iPerf, TCP Statistics, and Congestion Control

Group member	Name	Purdue ID
1	Kurt Khaustov	kkhausto
2	Kush Nayak	nayakk
3	Nicholas Kann	nkann
4	Aun Ali	ali221
5	Milosz Fecko	mfecko

Repository: github.com/aunali0/CS422_assignment_2. Every code link in this report points to the exact commit the report was generated from. This PDF is generated by `python main.py report` (report.py) from `REPORT.md` and the saved measurements, so every figure and table is reproducible from the repository.

1. iPerf throughput application

1(a) Client implementation

The client implements the iperf3 protocol directly with Python sockets; it never runs the iperf3 binary. run_test performs one upload test against a public iperf3 server:

- **Control connection.** Connect with a 5 s timeout and send a 37-byte cookie: 36 random lowercase letters plus a NUL (cookie). Each protocol state is one signed byte; receive_state checks that every byte is the expected state.
- **Parameter exchange.** After `PARAM_EXCHANGE` (9), send the test parameters as a 4-byte big-endian length followed by JSON (send_json): TCP, one stream, the test duration, and a 128 KiB block length.
- **Data connection.** After `CREATE_STREAMS` (10), open a second TCP connection (data_connection), selecting the congestion control algorithm before `connect()` when one is requested, and send the same cookie.
- **Transfer.** After `TEST_START` (1) and `TEST_RUNNING` (2), send 128 KiB random blocks until the configured duration (`-t`) ends, sampling TCP statistics every interval (`-g`) (transfer loop).
- **Termination.** Send `TEST_END` (4), wait for `EXCHANGE_RESULTS` (13), exchange result JSON (the server's reply is validated by a strict pydantic model in receive_json), wait for `DISPLAY_RESULTS` (14), and finish with `IPERF_DONE` (16) (termination.

Robustness. Every failure becomes an exception that try_run_test logs and turns into "no result", so the caller can try a replacement server:

- *Non-responsive servers and connection timeouts:* connect timeout (5 s), a deadline for each control message (receive_bytes, 3 s), and a data-socket idle timeout (3 s without progress).
- *Overloaded servers and rejected negotiation:* any unexpected state, including `ACCESS_DENIED` (−1, server busy) and `SERVER_ERROR` (−2), raises an error naming the state. JSON replies are bounded to 1 MiB and must match the expected schema.
- *Premature termination:* an empty `recv()` or a broken pipe while sending ends the test with an error instead of hanging.

The saved comparison experiment exercised these paths: the failure causes in Table 1 come from its log.

1(b) Destination selection

`python main.py destinations -s iperf3-servers.csv -c N -a A` tests `N` random destinations (default 10) from the server list of iperf3serverlist.net (run_destinations). The list is shuffled; each attempt takes the next untried server, so no server is tested twice. A server that fails is logged and replaced, with at most `A` attempts per destination (default 3). The command reports any shortfall (for example "Selected 7/10 servers (shortfall: 3)"). Each successful test records its host and port in `test-metadata.json`.

The comparison experiment used in this report (`compare`, Q3) selects destinations the same way, with a seeded shuffle (seed 422) and at most `count × attempts` candidates (run_comparison). A destination is kept only if all 9 of its tests (3 algorithms × 3 repetitions) succeed; otherwise the next candidate replaces it. `manifest.json` records every candidate, test, status, and order. Reaching 3 complete destinations took 8 candidates, 72 tests, and 23 failures:

Candidate (in order)	Passed	Failed	Failure causes	Used
dal.speedtest.clouvider.net:5200	9	0	–	yes
185.152.66.67:5201	0	9	ACCESS_DENIED ×9	no (incomplete)
a110.speedtest.wobcom.de:5201	2	7	ACCESS_DENIED ×4, TimeoutError ×1, BrokenPipeError ×1, OSError ×1	no (incomplete)
speedtest.wdc2.us.leaseweb.net:5201	8	1	BrokenPipeError ×1	no (incomplete)
nl.speed.vps1.net:5201	5	4	TimeoutError ×4	no (incomplete)
speed.netfiber.net.il:5201	9	0	–	yes
iperf.pendc.com:5201	7	2	TimeoutError ×2	no (incomplete)
96.45.41.167:30001	9	0	–	yes

Table 1. Every candidate destination in selection order, test outcomes, and failure causes from `experiment.log`*. "Used" destinations completed all 9 tests.*

1(c) Goodput

Goodput is computed from the kernel's cumulative acknowledged-byte counter on the data socket, read with `getsockopt(IPPROTO_TCP, TCP_INFO)` at every sampling interval (take_sample):

```
goodput(t) = (tcpi_bytes_acked[t] - tcpi_bytes_acked[t-1]) / (t - (t-1)) * 8   [bit/s]
```

`t − (t−1)` is the actual elapsed time between the two readings from a monotonic clock, not the nominal interval. Figure 1 and Table 2 use every successful 60-second test of the comparison experiment (1-second sampling): seven destinations, with three runs of each algorithm at the three complete destinations. Partial destinations are included here because Q1 concerns goodput per destination; they are excluded from the Q3 comparison.

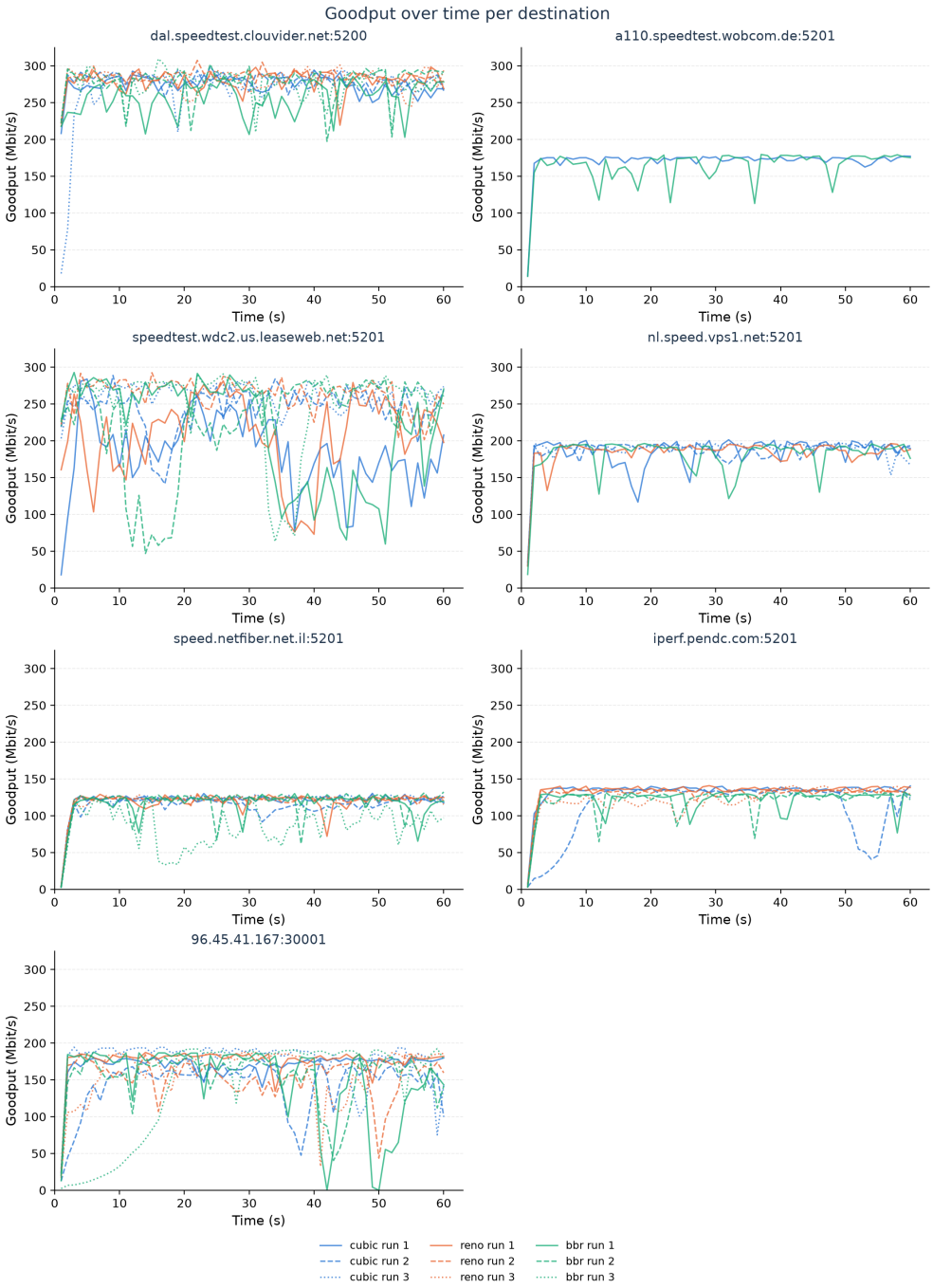


Figure 1. Goodput over time for every destination with at least one successful test, one line per run. Colors identify the algorithm and line styles the repetition. Generated by plot_destinations.

Destination	Algorithm	Runs	Min	Median	Mean	P95
dal.speedtest.clouvider.net:5200	cubic	3	17.66	279.39	273.96	292.22
dal.speedtest.clouvider.net:5200	reno	3	217.91	283.39	281.30	296.99
dal.speedtest.clouvider.net:5200	bbr	3	197.25	277.92	269.79	295.81
a110.speedtest.wobcom.de:5201	cubic	1	14.70	174.77	170.38	176.24
a110.speedtest.wobcom.de:5201	bbr	1	13.76	173.24	163.87	178.36
speedtest.wdc2.us.leaseweb.net:5201	cubic	3	17.74	248.05	230.50	279.04
speedtest.wdc2.us.leaseweb.net:5201	reno	2	72.87	250.65	235.58	284.81

Destination	Algorithm	Runs	Min	Median	Mean	P95
speedtest.wdc2.us.leaseweb.net:5201	bbr	3	46.73	262.50	229.38	284.11
nl.speed.vps1.net:5201	cubic	3	29.51	191.57	185.64	198.66
nl.speed.vps1.net:5201	reno	1	29.53	187.83	183.82	195.10
nl.speed.vps1.net:5201	bbr	1	18.26	189.43	180.35	195.18
speed.netfiber.net.il:5201	cubic	3	3.59	121.62	118.09	127.95
speed.netfiber.net.il:5201	reno	3	3.59	122.68	119.07	127.38
speed.netfiber.net.il:5201	bbr	3	2.18	119.63	106.59	128.92
iperf.pendc.com:5201	cubic	2	3.05	134.19	121.60	138.71
iperf.pendc.com:5201	reno	3	3.25	133.28	128.15	139.32
iperf.pendc.com:5201	bbr	2	2.85	126.81	120.55	131.10
96.45.41.167:30001	cubic	3	12.57	171.85	163.12	193.20
96.45.41.167:30001	reno	3	16.49	175.33	163.53	185.67
96.45.41.167:30001	bbr	3	0.00	172.38	147.47	189.97

Table 2. Goodput in Mbit/s per destination and algorithm, pooling the 1-second samples of all runs. P95 interpolates linearly at sorted index $(n - 1) \times 0.95$ (percentile, table).

Startup intervals and stalls are kept, which is why some minimums are near zero; the near-zero minimum for BBR at 96.45.41.167 is a stall discussed in Q3.

2. TCP statistics tracing

All Q2 measurements use CUBIC, the host's default algorithm, read back from the data socket with `getsockopt(TCP_CONGESTION)` and recorded with every sample. The host kernel was `6.18.54.1-microsoft-standard-WSL2` (WSL2).

2(a) Extraction, storage, and decoding

At every goodput sample the client reads `TCP_INFO` from the **data socket** (never the control socket), so goodput and the TCP statistics share one timestamp (take_sample). save_run writes one CSV row per sample to `tcp-samples.csv`, with the run ID, destination, host, port, and algorithm on every row, and writes `test-metadata.json` with the kernel version, the number of `TCP_INFO` bytes returned, the algorithm read back, and the unit of every column.

- **Recorded per sample:** timestamp (seconds since the upload started), interval length, goodput, `snd_cwnd`, `snd_ssthresh`, smoothed RTT (`rtt`), `rttvar`, `min_rtt`, pacing rate, delivery rate, cumulative byte and segment counters, and every other decoded `tcp_info` field (59 fields, prefixed `tcpi_`).
- **Decoding format.** tcp_info.py decodes Linux `struct tcp_info` from `include/uapi/linux/tcp.h`: 59 fields in declaration order, native byte order, no padding (`struct` format `"="`, 248 bytes). Our kernel returned 280 bytes; the 32 bytes of newer trailing fields are ignored, and a shorter buffer from an older kernel leaves missing fields empty instead of failing (parse_tcp_info). The decoded fields were checked against `ss -ti` for the same socket.
- **Interval measurements.** Cumulative counters (`bytes_acked`, `total_retrans`, `bytes_retrans`, `data_segs_out`, `delivered`) are differenced between consecutive readings and, for rates, divided by the measured elapsed time. Instantaneous values (`snd_cwnd`, `snd_ssthresh`, `rtt`, `rttvar`, `lost`, pacing and delivery rate) are the values at the end of the interval.

- **Loss proxy.** TCP does not expose per-interval losses, so the loss signal is the number of segments retransmitted during the interval, `Δ tcpi_total_retrans`. It also counts unnecessary retransmissions, so it is a proxy rather than a loss rate. The CSV also stores `Δ tcpi_bytes_retrans` and the retransmitted share of data segments. `tcpi_lost` is the kernel's current estimate during recovery, so it is stored as-is and never differenced.

Field	Offset (bytes)	Format	Unit
tcpi_snd_mss	16	u32	bytes
tcpi_unacked	24	u32	segments
tcpi_lost	32	u32	segments
tcpi_rtt	68	u32	us
tcpi_rttvar	72	u32	us
tcpi_snd_ssthresh	76	u32	segments
tcpi_snd_cwnd	80	u32	segments
tcpi_total_retrans	100	u32	segments
tcpi_pacing_rate	104	u64	bytes/s
tcpi_bytes_acked	120	u64	bytes
tcpi_notsent_bytes	144	u32	bytes
tcpi_min_rtt	148	u32	us
tcpi_data_segs_out	156	u32	segments
tcpi_delivery_rate	160	u64	bytes/s
tcpi_delivered	192	u32	segments
tcpi_bytes_retrans	208	u64	bytes

Table 3. Byte offsets of the fields this project uses within the 248-byte decoded prefix, computed from the decoder's own field table.

2(b) Visualization

The representative destination is `185.180.12.40:5201` (DataPacket, Vienna; about 127 ms minimum RTT), tested with `python main.py test -s 185.180.12.40 -t 15 -g 0.5` (15 s, 0.5 s sampling). We chose this run because it contains several loss episodes, so the congestion-avoidance response is visible. A run to the same server three minutes earlier held a steady 176 Mbit/s with no retransmissions. Both runs are saved under `results/single-server-tests/`.

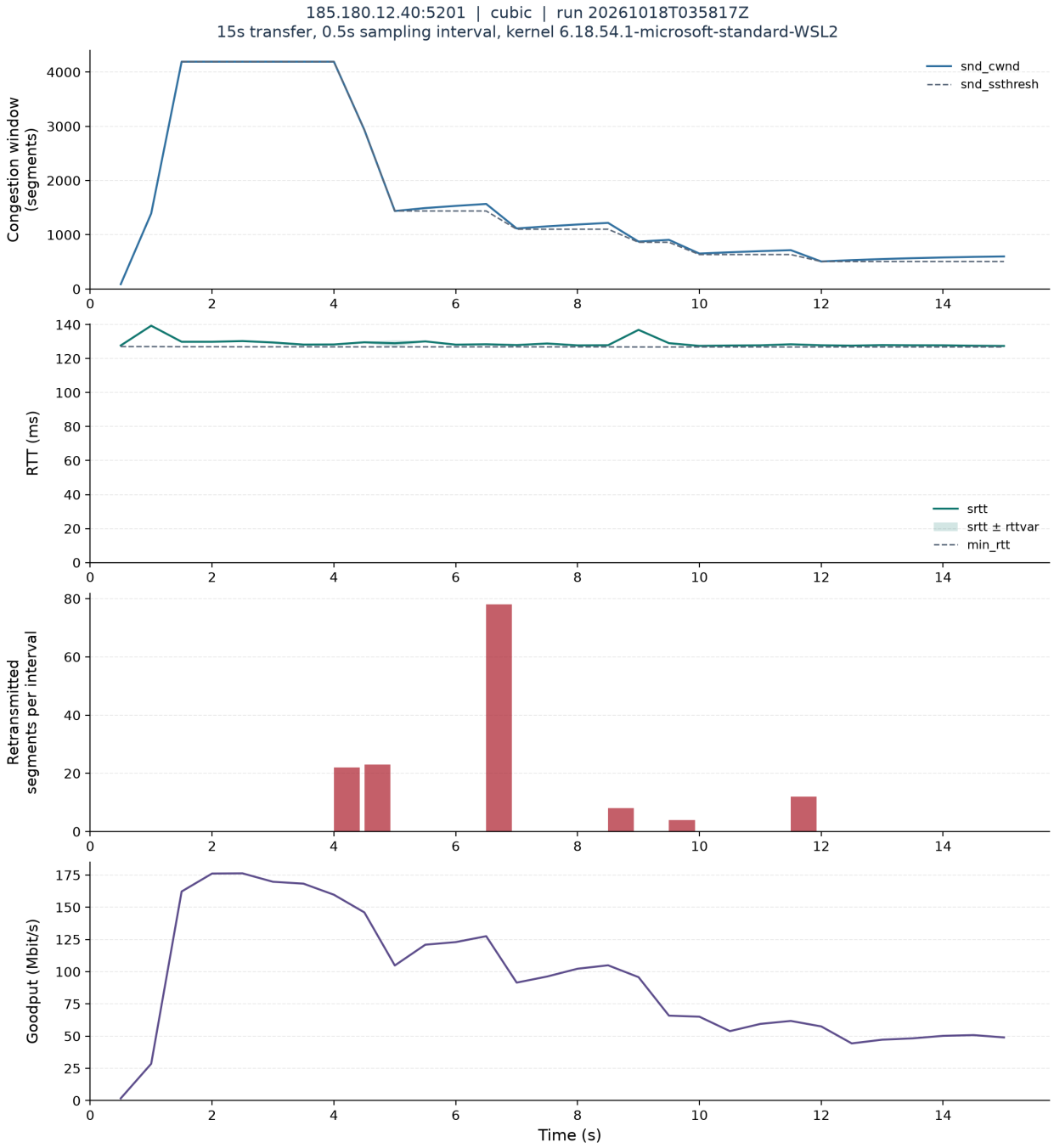


Figure 2. Q2(b)(i) time series for the representative destination (CUBIC): congestion window and slow-start threshold, smoothed RTT ± rttvar with the minimum RTT, retransmitted segments per interval (loss proxy), and goodput. Generated by plot_time_series.

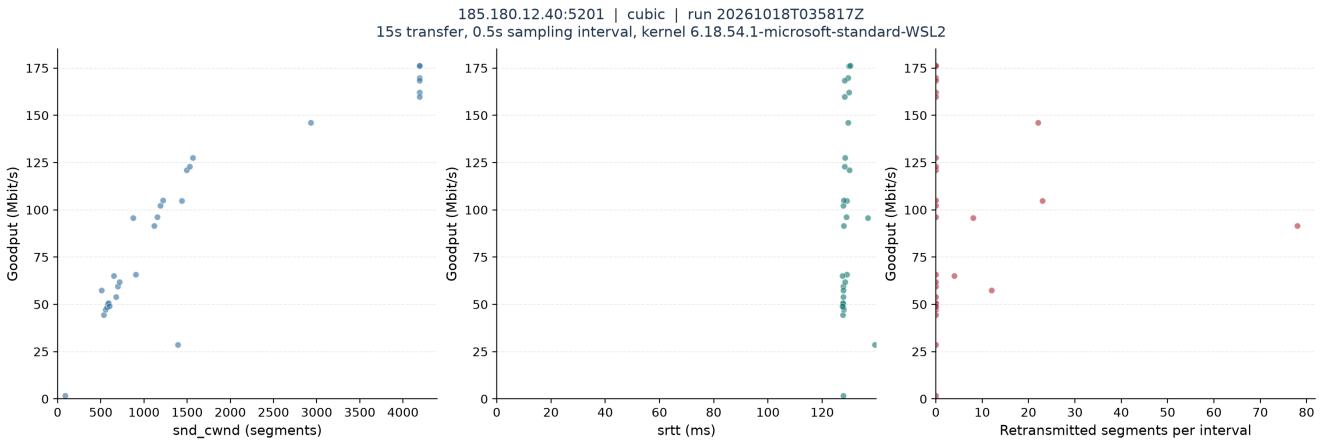


Figure 3. Q2(b)(ii) congestion window, smoothed RTT, and retransmitted segments per interval, each against the goodput of the same interval. Generated by plot_scatter.

2(c) Observations

What the metrics mean and how they should affect goodput in congestion avoidance.

- *Congestion window (*`snd_cwnd`*)* is the maximum number of unacknowledged segments the sender may have in flight. When the window is the limit, goodput ≈ cwnd × MSS / RTT, so goodput should rise and fall with cwnd. CUBIC grows the window along a cubic curve between losses and multiplies it by β = 0.7 after a loss [2]. `snd_ssthresh` is the boundary between slow start (doubling per RTT) and congestion avoidance.
- *Smoothed RTT (*`srtt`*)* is the kernel's moving average of RTT samples, and `rttvar` its variation. The lowest RTT approximates the propagation delay; RTT above it means queueing. For a fixed window, a higher RTT means lower goodput, and RTT rising while goodput stays flat indicates a queue building at the bottleneck.
- *Loss proxy (retransmitted segments per interval):* each loss episode makes CUBIC reduce its window, so retransmissions should be followed by lower goodput that recovers slowly as the window grows again.
- *Goodput* is the rate at which application data is acknowledged: the outcome the other three metrics explain.

What the trace shows.

- *Slow start ended without loss.* The window grew from 88 to 4,191 segments in 1.5 s, then `snd_ssthresh` was set equal to cwnd with zero retransmissions. This matches Linux CUBIC's HyStart, which leaves slow start when it detects RTT growth instead of waiting for a loss.
- *At first, the window was not the limit.* From 1.5 s to 4 s, cwnd stayed at 4,191 segments, enough for about 355 Mbit/s at 128 ms, yet goodput was 160–176 Mbit/s. `tcpi_unacked` shows only about 2,013 segments in flight and almost no data waiting in the socket (`tcpi_notsent_bytes` < 30 KB), so our sender was not keeping the window full: the limit was on our side (application or send buffer), not the network. Linux does not grow cwnd while the window is unused, which is why cwnd is flat.
- *Each loss cut the window by CUBIC's β.* Retransmissions at 4.5, 5.0, 7.0, 9.0, 10.0, and 12.0 s each reduced `snd_ssthresh` to 0.70 × the previous cwnd (4,191 → 2,934 → 1,438 after two episodes, 1,568 → 1,102, 1,219 → 861, 906 → 635, 716 → 506).
- *After the losses, goodput followed the window.* From 5 s on, in-flight segments equal cwnd and more than 600 KB waits in the socket, so the flow is window-limited, and goodput matches cwnd × MSS / RTT within a few Mbit/s (121 vs 125, 62 vs 61, 48 vs 49 Mbit/s). The cwnd–goodput scatter therefore falls on a straight line.
- *Anomaly: losses without queueing delay.* After slow start, smoothed RTT stayed at 127–130 ms, within 3 ms of the minimum (one 137 ms sample), and the RTT–goodput scatter is a vertical band. Losses therefore did not

come from a queue that filled and raised delay; a shallow buffer, a rate policer, or random loss on the path would produce this pattern. CUBIC treated each loss as congestion, so goodput fell from 176 to about 50 Mbit/s.

- *Anomaly: slow recovery.* Between losses the window grew by only 8–54 segments per 0.5 s (at most about 14 segments per RTT, slowing as it approached its previous size, as CUBIC's concave growth predicts), so after the last loss at 12 s goodput was still near 50 Mbit/s when the 15-second test ended. A 15-second test at 127 ms RTT is short for CUBIC to regain its earlier window.
- `tcpi_lost` and `tcpi_ca_state` read 0 / Open at almost every sample, because each recovery lasted about one RTT, much less than the 0.5 s sampling interval; the cumulative retransmission counter is what captures these events.

3. Comparing algorithms and designing one

3(a) CUBIC, Reno, and BBR

We compared CUBIC, Reno, and BBR at three destinations with three 60-second runs per algorithm and 1-second sampling: 27 matched runs (1,620 samples), using the same destinations, duration, and sampling interval for every algorithm. Each algorithm was set on the data socket before `connect()` and read back after connecting (data_connection); a mismatch fails the test. A preflight check confirms every algorithm is available before contacting servers (preflight). The host ran Linux `6.18.33.2-microsoft-standard-WSL2`, Python 3.14.8, matplotlib 3.11.2, and pydantic 2.13.5.

Destinations were shuffled with seed 422 (Table 1). The algorithm order was shuffled once and then rotated by repetition and destination, so each algorithm ran first, second, and third once per destination (Table 4). Tests ran one at a time with a 2-second pause. Dallas, the first destination to complete, is the representative for Figures 4 and 5; it was not chosen for its throughput.

Destination	Repetition 1	Repetition 2	Repetition 3
dal.speedtest.clouvider.net:5200	bbr → cubic → reno	cubic → reno → bbr	reno → bbr → cubic
speed.netfiber.net.il:5201	reno → bbr → cubic	bbr → cubic → reno	cubic → reno → bbr
96.45.41.167:30001	cubic → reno → bbr	reno → bbr → cubic	bbr → cubic → reno

Table 4. Test order per destination and repetition, from `manifest.json`.

Goodput per run is all acknowledged bytes divided by the run's elapsed time. Table 5 averages the three runs; ± is the sample standard deviation across runs. RTT is the mean sampled smoothed RTT; retransmissions are totals over the three runs and are a loss proxy, not a loss probability (run_statistics). Per-run minimum, median, mean, and P95 goodput are in `reports/per-test-statistics.csv`.

Destination	Algorithm	Goodput (Mbit/s)	Mean RTT (ms)	Retrans. (segs)
dal.speedtest.clouvider.net:5200	cubic	273.96 ± 5.63	68.73	224
dal.speedtest.clouvider.net:5200	reno	281.30 ± 2.76	73.65	444
dal.speedtest.clouvider.net:5200	bbr	269.79 ± 11.69	64.53	196
speed.netfiber.net.il:5201	cubic	118.08 ± 4.16	181.90	23
speed.netfiber.net.il:5201	reno	119.07 ± 0.89	181.67	41
speed.netfiber.net.il:5201	bbr	106.59 ± 17.37	177.39	398
96.45.41.167:30001	cubic	163.12 ± 17.51	132.58	120
96.45.41.167:30001	reno	163.52 ± 11.80	136.69	11
96.45.41.167:30001	bbr	147.47 ± 6.02	150.60	541

Table 5. Mean results across three repetitions per algorithm. Dallas is `dal.speedtest.clouvider.net:5200`*; Netfiber is* `speed.netfiber.net.il:5201`*.*

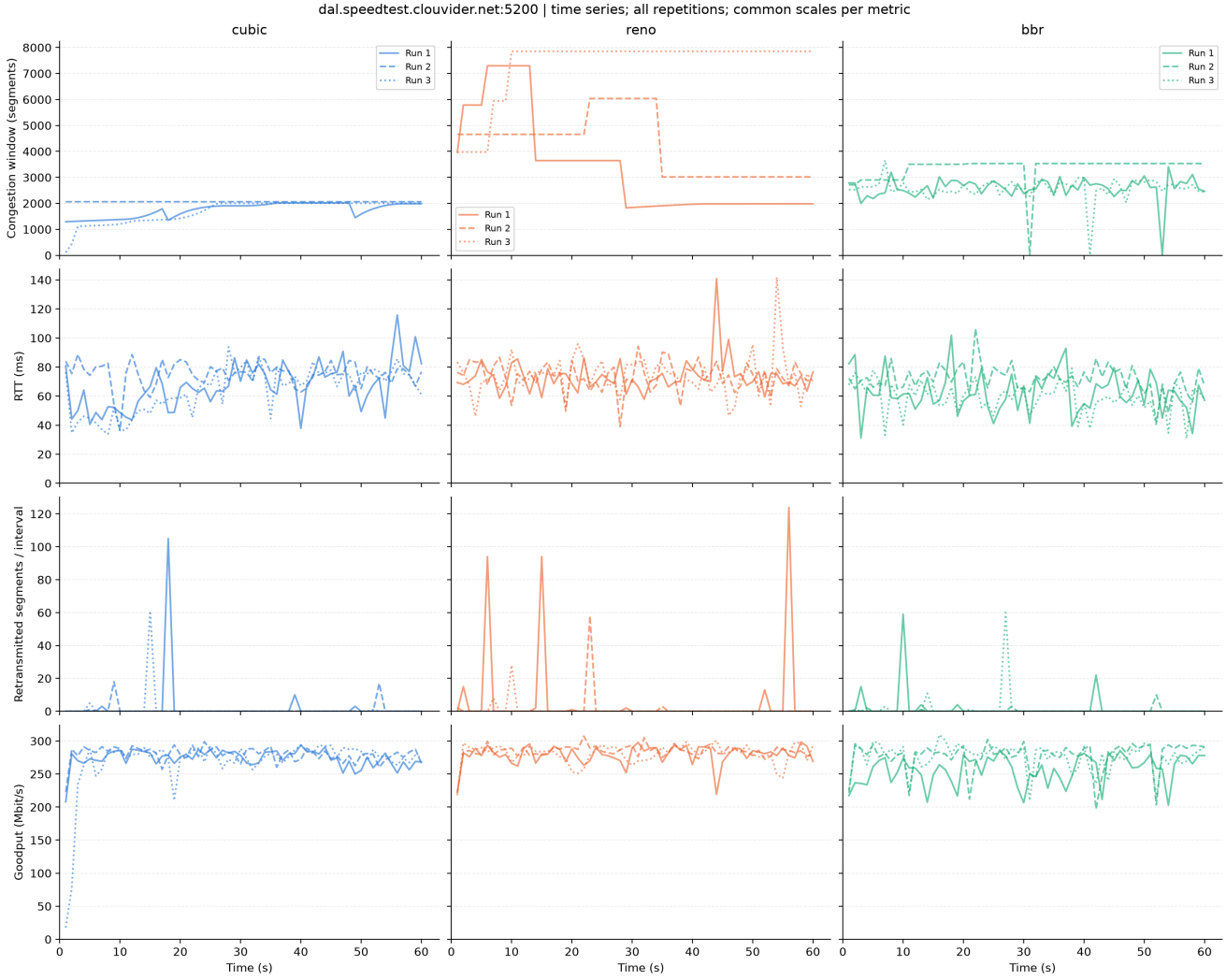


Figure 4. Q3(a) time series at the representative destination (Dallas): congestion window, smoothed RTT, retransmitted segments per interval, and goodput for each algorithm. Each row uses one scale for all algorithms; line styles distinguish repetitions. Generated by plot_comparison.

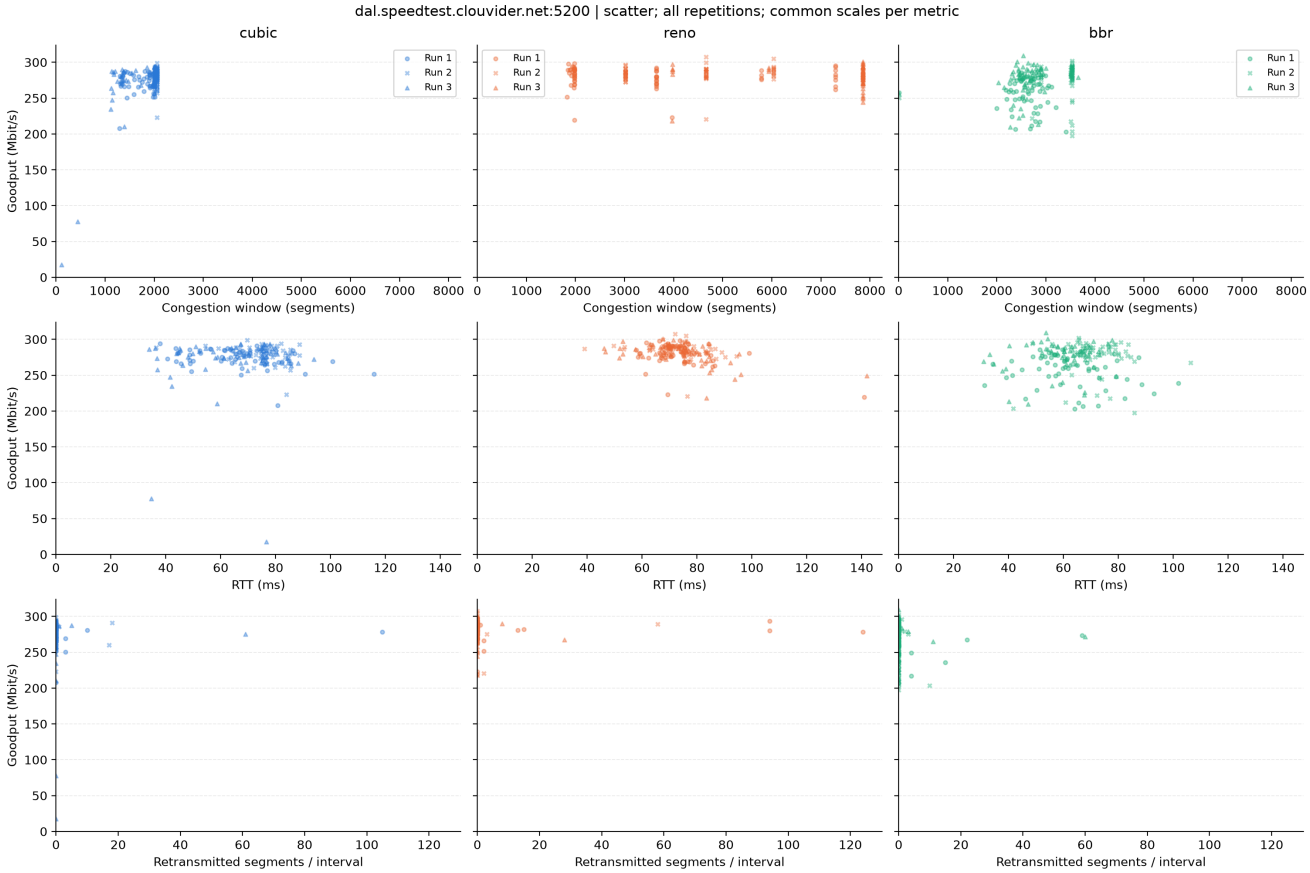


Figure 5. Q3(a) congestion window, RTT, and retransmissions against goodput at Dallas, all repetitions, with common scales per metric.

Observations.

- *Dallas (about 30 ms minimum RTT, around 280 Mbit/s):* Reno averaged 281.30 Mbit/s, CUBIC 273.96, and BBR 269.79. BBR had the lowest mean RTT (64.53 ms) and Reno the highest (73.65 ms), so Reno's small throughput advantage came with more delay. Reno's median cwnd rose from 1,982 to 7,854 segments across runs while goodput stayed at 279–284 Mbit/s: the extra window did not deliver more data. Minimum RTT was about 29–32 ms, well below the sampled means, consistent with queueing or other variable delay.
- *Netfiber (about 170–174 ms minimum RTT):* Reno and CUBIC were close (119.07 vs 118.08 Mbit/s) with mean RTTs near 182 ms. BBR had slightly lower RTT but averaged 106.59 Mbit/s: its third run fell to 86.57 Mbit/s with 396 retransmissions, versus two and zero in its first two runs. RTT stayed near its baseline during much of that slowdown, so RTT alone would not have revealed the problem.
- *96.45.41.167:30001:* Reno and CUBIC were effectively tied (163.52 vs 163.12 Mbit/s); their run standard deviations (11.80 and 17.51) are far larger than the difference. BBR averaged 147.47 Mbit/s with the most retransmissions (541). About 50 s into its first run, one interval delivered nothing while RTT reached 1,559.64 ms; the transfer then recovered and completed. The stall is kept in the statistics.

Operating regimes. On these single-flow paths, loss-based Reno and CUBIC delivered the most throughput, and Reno's linear growth was enough to fill the windows these paths needed. BBR traded some throughput for lower RTT on the short-RTT Dallas path (it paces at its estimate of bottleneck bandwidth and probes RTT, keeping queues shorter [3]), but it lost the most throughput on the two long-RTT paths, where it also retransmitted most. CUBIC's faster window growth [2] should matter most on long, high-bandwidth paths, but these paths needed no more window than Reno already had, so it showed no advantage.

Window, queueing, and the bandwidth-delay product. For a window-limited flow, rate ≈ cwnd × MSS / RTT, and the bandwidth-delay product (BDP, bandwidth × propagation RTT) is the data needed in flight to fill the path. Using achieved goodput × minimum RTT as an estimate, Dallas's third Reno run needed about 1.09 MB in flight but had a median window of about 11.37 MB. A window ten times the BDP does not raise throughput; the excess either sits in queues (raising RTT, as Reno's higher RTT suggests) or goes unused because something else limits the flow. Retransmissions mark where loss-based algorithms found a full queue or a policer and backed off.

Variability and limitations. Public paths and cross-traffic changed between tests, and rotating the order spreads but does not remove that variation. Run-to-run spread was often larger than the differences between algorithms, so three runs with 1-second samples support only the clear differences above. Keeping only destinations where all nine tests succeeded favors reliable servers. Server-side limits, receiver windows, and our own send path can cap throughput. These single-flow tests say nothing about fairness between competing flows.

3(b) Proposed congestion control algorithm

The controller uses three signals with distinct roles: loss triggers a strong reduction through the TCP stack's recovery; low RTT allows growth; high RTT without improving goodput triggers a small reduction. Start with cwnd = 10 segments and an unlimited slow-start threshold. Keep the minimum valid RTT since the connection started and the previous eligible interval's goodput (newly acknowledged bytes / actual elapsed seconds). Make one window decision per RTT with new ACK progress. The full specification is `Q3_DESIGN.md`.

```
State: slow_start = true; previous_goodput = unset
On a new loss-recovery episode, once:
    request ssthresh = max(2, floor(cwnd / 2))
    let Linux apply recovery using that threshold
    slow_start = false; clear previous_goodput and interval accounting
On a retransmission timeout:
    let Linux apply its normal timeout window and recovery
    slow_start = true; clear previous_goodput and interval accounting
Once per RTT with new ACKs, outside recovery:
    update minimum_RTT and measure goodput
    if application limited or receive-window limited:
        clear previous_goodput; finish interval
    if slow_start and cwnd < ssthresh and RTT <= 1.10 * minimum_RTT:
        cwnd += min(newly_acked_segments, cwnd)
    else:
        slow_start = false
        if RTT > 1.25 * minimum_RTT and previous_goodput is valid
           and goodput <= previous_goodput:
            cwnd = max(2, floor(0.90 * cwnd)); ssthresh = cwnd
        else if RTT <= 1.10 * minimum_RTT:
            cwnd += 1
        else:
            hold cwnd
    obey kernel limits; previous_goodput = goodput; finish interval
```

Design choices. Each rule answers something we measured. At Dallas a much larger window did not raise goodput, so when RTT is high the controller asks whether goodput is still improving before letting the window stay large. At Netfiber BBR's slow run had many retransmissions with RTT near its baseline, so loss gets its own, stronger response independent of RTT. On the IP path a stall came with a large RTT increase, which the delay rule responds to. The band between 10% and 25% RTT inflation is a hold zone that avoids reacting to small RTT noise, and the 10% delay reduction is deliberately milder than the 50% loss reduction because delay is an earlier, softer signal. Slow start exits on delay as well as loss, like the HyStart behavior seen in Q2. These thresholds are starting values to test, not tuned results.

Fairness and convergence. Assuming long-running flows with equal RTTs share one bottleneck, additive increase with multiplicative decrease on loss moves competing windows toward equal shares, as in Reno. The delay rule also reduces every flow when the shared queue grows, which helps both fairness and lower queueing. These are not

guaranteed: flows with shorter RTTs update more often and get more bandwidth; this delay-sensitive controller will yield to Reno or CUBIC flows that keep the queue full; a minimum RTT recorded before a route change keeps the controller too cautious; and per-interval goodput is noisy. Repeated increases and decreases can settle into a cycle around the operating point rather than a fixed rate. The design has not been implemented or evaluated; Assignment 3 must compare it with the three baselines.

Reproduction

Inputs, raw data, and outputs are in the repository: the server list `iperf3-servers.csv`, raw samples and metadata under `results/`, and pinned dependencies in `requirements.txt` (Python 3.12+, matplotlib 3.11.2, pydantic 2.13.5, reportlab 5.1.0). The single entry point is `main.py`; the README documents every option, including the number of destinations (`-c`), transfer duration (`-t`), sampling interval (`-g`), and repetitions (`-r`).

```
python -m pip install -r requirements.txt
python main.py test -s 185.180.12.40 -p 5201 -t 15 -g 0.5            # Q2 trace
python main.py destinations -s iperf3-servers.csv -c 10 -a 3 -t 60 -g 1  # Q1
python main.py compare -s iperf3-servers.csv -c 3 -t 60 -g 1 -r 3 \
    --seed 422 --run-id my-run                                      # Q3
python main.py analyze results/algorithm-comparisons/q3-test          # rebuild plots offline
python main.py report                                                 # rebuild this PDF
```

Docker (`FROM ubuntu:24.04`, `Dockerfile` in the repository) runs the full pipeline:

```
docker build -t cs422-a2 .
docker run --rm --network host --cap-add NET_ADMIN -v "$PWD/results:/results" \
    cs422-a2 compare -s iperf3-servers.csv -c 3 -t 60 -g 1 -r 3 --run-id docker-run -o /results
```

The container uses the host kernel, so TCP behavior and `TCP_INFO` depend on the host. Required host configuration: Linux or WSL2 with `cubic`, `reno`, and `bbr` available (`sudo modprobe tcp_bbr`). Selecting an algorithm outside `net.ipv4.tcp_allowed_congestion_control` needs `CAP_NET_ADMIN`, hence `--cap-add NET_ADMIN` (or add `bbr` to that sysctl). Host kernels: Q2 `6.18.54.1-microsoft-standard-WSL2`, Q3 `6.18.33.2-microsoft-standard-WSL2`; each test also records its kernel in `test-metadata.json`.

References

[1] RFC 5681, TCP Congestion Control, §3.1 (Reno): rfc-editor.org/rfc/rfc5681

[2] RFC 9438, CUBIC for Fast and Long-Distance Networks: rfc-editor.org/rfc/rfc9438

[3] Google BBR documentation: github.com/google/bbr

AI acknowledgement

We used AI assistants (Claude and others) to help write and refactor code (protocol handling, TCP_INFO decoding, plotting, the experiment runner, and this report generator), to check measurements against the raw data, and to draft and edit report text. A teammate collected all measurements on Linux/WSL2. We reviewed the code and verified the reported numbers against the saved CSV files.